DOKUMENTASI PROYEK · UNTUK PENGEMBANG DAN PENGELOLA SISTEM

Dokumentasi Teknis LabTrack

Arsitektur, konfigurasi, API, infrastruktur, CI/CD, operasi, dan pemulihan

Versi aplikasi: 1.0.2 (commit 81840b2)

Proyek Google Cloud: labtrack-1026146, region asia-southeast2

Repositori: github.com/Muhammadabrarrayhan2/labtrack

Pengelola: [Nama lengkap peserta]

Pembaruan terakhir: 26 Oktober 2026

Daftar Isi

1. Tentang dokumen ini 3
2. Arsitektur 3
    2.1 Akun layanan dan izin 4
    2.2 Model data 5
3. Menjalankan di lingkungan lokal 5
    3.1 Variabel lingkungan 6
4. API 6
    4.1 Format galat 7
    4.2 Contoh kehadiran massal (data contoh) 7
5. Antarmuka web 8
6. Infrastruktur sebagai kode 8
    6.1 Berkas Terraform 8
    6.2 Variabel yang mengendalikan biaya 9
    6.3 Perintah dasar 9
7. CI/CD 9
8. Operasi 10
    8.1 Jadwal rutin 10
9. Pemantauan dan SLA 11
10. Pemulihan bencana 12
11. Keamanan 12
12. Batasan yang diketahui 13

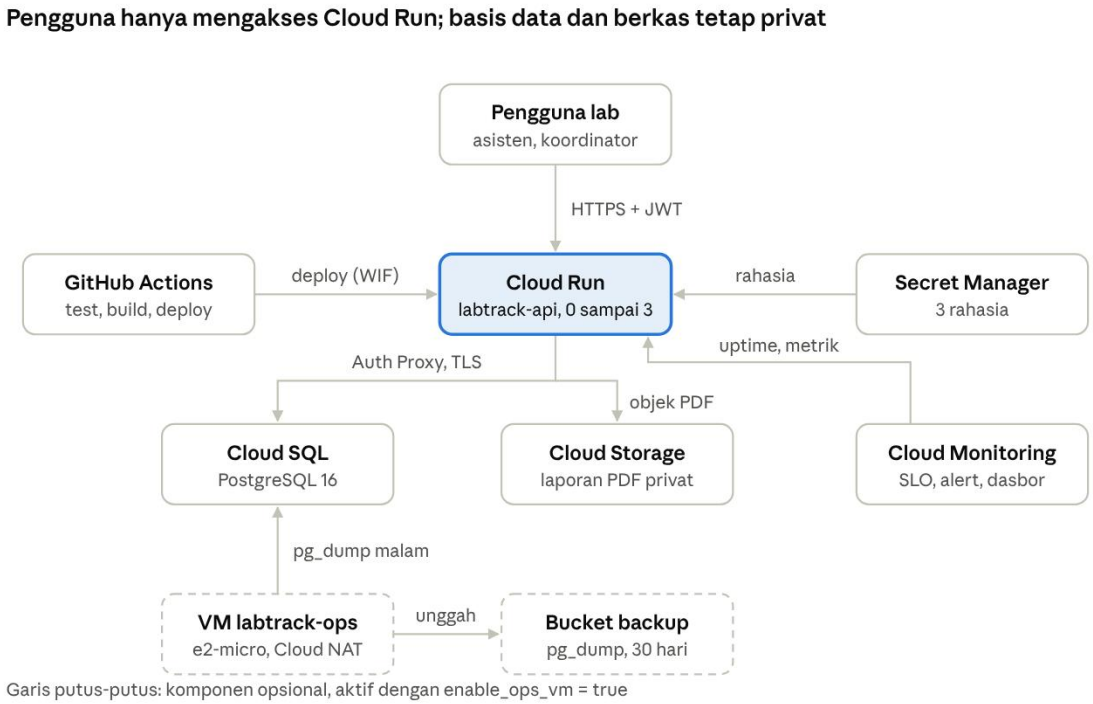
1. Tentang dokumen ini

Dokumen ini ditujukan untuk orang yang akan memelihara LabTrack: memperbaiki kode, menambah fitur, menjalankan rilis, atau menangani gangguan. Isinya merangkum keputusan teknis, cara menjalankan sistem di laptop dan di Google Cloud, serta prosedur operasi yang sudah diuji. Penjelasan yang lebih panjang per unit kompetensi ada di folder `docs/` di repositori, dan dokumen ini merujuk ke sana bila perlu.

Tabel 1. Riwayat versi

Versi	Tanggal	Perubahan
1.0.0	25 Oktober 2026	Rilis pertama: API, Terraform, CI/CD, pemantauan SLO
1.0.1	26 Oktober 2026	Toleransi selisih jam 30 detik pada validasi JWT untuk mengatasi galat 401 antarinstans
1.0.2	26 Oktober 2026	Antarmuka web untuk staf di `/app/`; peramban yang membuka `/` diarahkan ke sana

2. Arsitektur



Gambar 1. Komponen LabTrack di Google Cloud

Tabel 2. Komponen dan konfigurasinya

Komponen	Sumber daya	Konfigurasi penting
Aplikasi	Cloud Run v2 `labtrack-api`	0 sampai 3 instans, konkurensi 40, 1 vCPU, 512 MiB, probe startup dan liveness ke `/health`
Basis data	Cloud SQL `labtrack-db-f6f7`	PostgreSQL 16, `db-f1-micro`, disk 10 GB bertambah otomatis, cadangan otomatis 01.00 WIB disimpan 7 buah, hanya koneksi terenkripsi

Komponen	Sumber daya	Konfigurasi penting
Berkas laporan	Bucket Cloud Storage privat	Akses seragam tingkat bucket, unduh lewat URL bertanda tangan 15 menit
Rahasia	Secret Manager	`labtrack-db-password`, `labtrack-jwt-secret`, `labtrack-admin-password`
Citra	Artifact Registry `labtrack`	Format Docker, region asia-southeast2
CI/CD	Workload Identity Pool dan Provider	Hanya repositori `Muhammadabrarrayhan2/labtrack` yang boleh memakai identitas deploy
VM operasi (opsional)	Compute Engine `labtrack-ops`	e2-micro, Debian 12, us-central1-a, tanpa IP publik, IAP SSH, Cloud NAT, snapshot harian 7 hari
Bucket cadangan (opsional)	Cloud Storage	Hasil `pg_dump` malam, dihapus otomatis setelah 30 hari
Pemantauan	Cloud Monitoring	2 SLO, 6 kebijakan alert, uptime check, dasbor 8 widget

2.1 Akun layanan dan izin

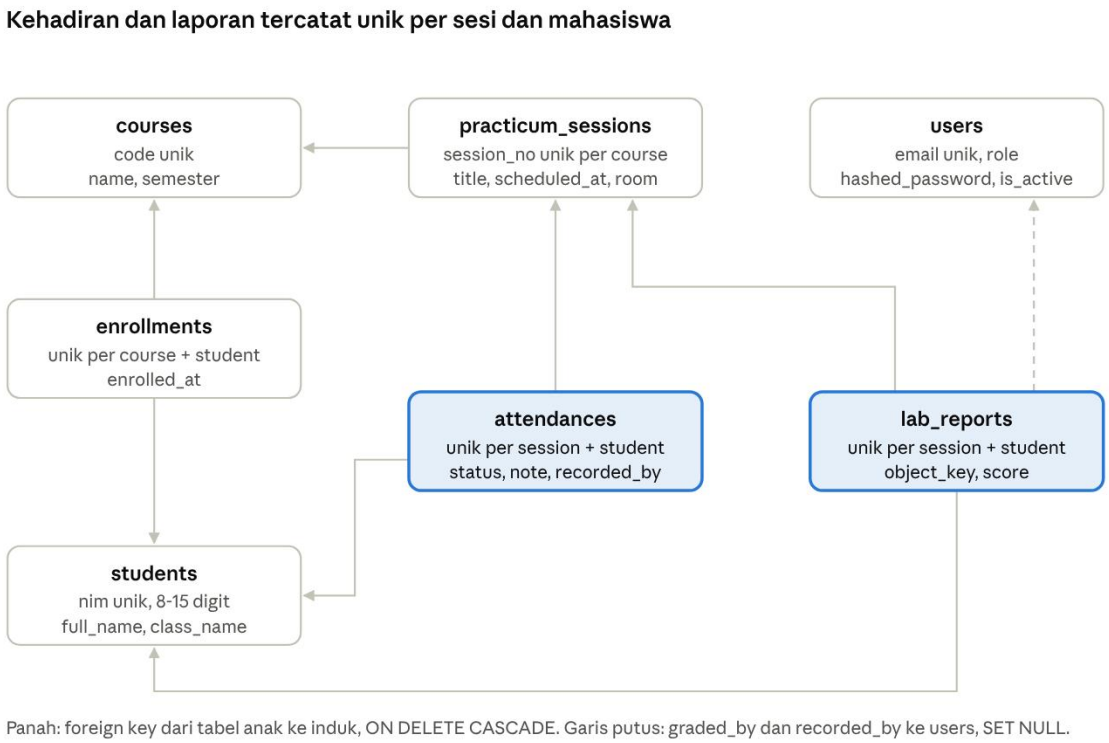
Ada tiga akun layanan, masing-masing dengan izin sesempit mungkin.

Tabel 3. Akun layanan

Akun layanan	Dipakai oleh	Peran
`labtrack-run`	Kontainer Cloud Run	`cloudsql.client`, penulis log dan metrik, `secretAccessor` pada tiga rahasia, `storage.objectAdmin` pada bucket laporan, `iam.serviceAccountTokenCreator` pada dirinya sendiri untuk menandatangani URL
`labtrack-deployer`	GitHub Actions lewat WIF	`run.developer`, `artifactregistry.writer` pada repositori citra, `iam.serviceAccountUser` pada `labtrack-run`
`labtrack-ops`	VM operasi	`cloudsql.client`, penulis log dan metrik, `secretAccessor` pada kata sandi basis data, `storage.objectCreator` pada bucket cadangan

Akun `labtrack-ops` sengaja tidak bisa membaca atau menghapus isi bucket cadangan. Bila VM itu disusupi, penyerang tidak bisa menghapus cadangan lama.

2.2 Model data



Gambar 2. Relasi tabel

Tabel 4. Tabel basis data

Tabel	Kolom utama	Batasan
`users`	email, full_name, hashed_password, role, is_active	email unik
`students`	nim, full_name, email, class_name	nim unik, 8 sampai 15 digit
`courses`	code, name, semester	code unik
`enrollments`	course_id, student_id, enrolled_at	unik per pasangan course dan student
`practicum_sessions`	course_id, session_no, title, scheduled_at, room	session_no unik per course
`attendances`	session_id, student_id, status, note, recorded_by	unik per session dan student
`lab_reports`	session_id, student_id, object_key, size_bytes, score, feedback, graded_by	unik per session dan student

Kunci asing dari tabel anak ke induk memakai `ON DELETE CASCADE`. Kolom `recorded_by` dan `graded_by` mengarah ke `users` dengan `SET NULL`, sehingga data kehadiran dan nilai tetap ada walaupun akun staf dihapus.

3. Menjalankan di lingkungan lokal

```
git clone https://github.com/Muhammadabrarrayhan2/labtrack.git
cd labtrack
docker compose up --build        # API di http://localhost:8000, PostgreSQL 16 di kontainer
# atau tanpa Docker:
python -m venv .venv && .venv/Scripts/pip install -r requirements-dev.txt
.venv/Scripts/python -m uvicorn app.main:app --reload
```

Buka `http://localhost:8000/` di peramban untuk antarmuka web, atau `http://localhost:8000/docs` untuk Swagger UI. Admin pertama dibuat dari variabel `BOOTSTRAP_ADMIN_EMAIL` dan `BOOTSTRAP_ADMIN_PASSWORD` saat aplikasi mulai, bila email itu belum ada.

3.1 Variabel lingkungan

Tabel 5. Konfigurasi aplikasi

Variabel	Bawaan	Keterangan
`APP_ENV`	`local`	`production` mewajibkan `JWT_SECRET` diisi; aplikasi menolak mulai bila masih bawaan
`DATABASE_URL`	kosong	Bila diisi, dipakai langsung (misalnya SQLite untuk uji)
`INSTANCE_CONNECTION_NAME`	kosong	`project:region:instance`; bila diisi, koneksi lewat Cloud SQL Python Connector
`DB_USER`, `DB_PASSWORD`, `DB_NAME`	`labtrack`, kosong, `labtrack`	Kredensial basis data; di Cloud Run kata sandi dari Secret Manager
`JWT_SECRET`	nilai pengembangan	Kunci HMAC token; di Cloud Run dari Secret Manager
`JWT_EXPIRE_MINUTES`	`120`	Masa berlaku token
`BOOTSTRAP_ADMIN_EMAIL`, `BOOTSTRAP_ADMIN_PASSWORD`	kosong	Admin pertama
`GCS_BUCKET`	kosong	Bila kosong, berkas disimpan di folder lokal `LOCAL_UPLOAD_DIR`
`SIGNED_URL_MINUTES`	`15`	Masa berlaku tautan unduh
`MAX_UPLOAD_MB`	`10`	Batas ukuran PDF
`CORS_ORIGINS`	`*`	Daftar asal yang diizinkan, dipisah koma
`LOG_LEVEL`	`INFO`	Log keluar sebagai JSON satu baris agar terbaca Cloud Logging

4. API

Seluruh endpoint kecuali `/auth/login`, `/health`, `/`, dan berkas statis di `/app/` memerlukan header `Authorization: Bearer <token>`. Dokumentasi interaktif ada di `/docs` (Swagger UI) dan `/redoc`.

Tabel 6. Daftar endpoint

Metode	Jalur	Peran	Fungsi
POST	`/auth/login`	publik	Login (form OAuth2 password), mengembalikan JWT
GET	`/auth/me`	semua	Data pengguna yang sedang masuk
POST, GET	`/auth/users`	admin	Membuat dan melihat akun staf
GET, POST	`/students`	semua	Mencari (parameter `q`, `limit`, `offset`) dan menambah mahasiswa
GET, PATCH, DELETE	`/students/{id}`	semua; DELETE hanya admin	Membaca, mengubah, menghapus
POST, GET	`/courses`	POST	Mata kuliah

Metode	Jalur	Peran	Fungsi
		hanya admin	
POST	`/courses/{id}/enrollments`	semua	Mendaftarkan mahasiswa; aman bila diulang
GET	`/courses/{id}/students`	semua	Peserta mata kuliah
GET	`/courses/{id}/attendance-summary`	semua	Rekap per mahasiswa; parameter `threshold` (bawaan 75)
POST, GET	`/sessions`	POST hanya admin	Sesi praktikum; filter `course_id`
PUT, GET	`/sessions/{id}/attendance`	semua	Simpan kehadiran massal (upsert) dan baca
POST, GET	`/sessions/{id}/reports`	semua	Unggah PDF (multipart: `student_id`, `file`) dan daftar laporan
GET	`/reports/{id}/download-url`	semua	URL bertanda tangan 15 menit
PATCH	`/reports/{id}/grade`	semua	Nilai 0 sampai 100 dan umpan balik
GET	`/health`	publik	Status aplikasi dan koneksi basis data
GET	`/`	publik	Peramban diarahkan ke `/app/`; klien lain menerima JSON deskripsi layanan

4.1 Format galat

Semua galat memakai bentuk yang sama, dan header `X-Request-ID` selalu dikirim. Contoh respons ketika NIM sudah terdaftar:

```
HTTP/1.1 409 Conflict
X-Request-ID: 3f9c2a7d41e84b0c9a6e5d1f20b7c843

{"detail": "Student with NIM 2026010101 already exists", "request_id":
"3f9c2a7d41e84b0c9a6e5d1f20b7c843"}
```

Di Cloud Run, `request_id` diambil dari header `X-Cloud-Trace-Context`, sehingga nilainya sama dengan trace ID di Cloud Logging. Untuk mencari log satu permintaan: `jsonPayload.request_id="<kode>"`.

Tabel 7. Kode status yang dipakai

Kode	Arti di LabTrack
401	Token tidak ada, salah, atau kedaluwarsa; atau login gagal
403	Peran tidak cukup, misalnya asisten membuat mata kuliah
404	Data tidak ditemukan
409	Data unik sudah ada (NIM, kode mata kuliah, nomor sesi, email)
413	PDF lebih dari 10 MB
415	Berkas bukan PDF berdasarkan isinya
422	Isian tidak lolos validasi
503	`/health` tidak bisa menjangkau basis data

4.2 Contoh kehadiran massal (data contoh)

```
PUT /sessions/12/attendance
{"records": [
  {"student_id": 41, "status": "present"},
  {"student_id": 42, "status": "late", "note": "terlambat 10 menit"},
  {"student_id": 43, "status": "absent"}
```

```
]}
```

Nilai `status` yang diterima: `present`, `late`, `excused`, `absent`. Mengirim ulang payload yang sama menghasilkan data yang sama.

5. Antarmuka web

Antarmuka web berada di `app/static/` dan terdiri dari tiga berkas: `index.html`, `style.css`, dan `app.js`. Tidak ada proses build dan tidak ada pustaka pihak ketiga. FastAPI melayaninya lewat `StaticFiles` di jalur `/app`.

Beberapa catatan implementasi:

- Token disimpan di `sessionStorage`, bukan `localStorage`, supaya hilang saat tab ditutup. Ini lebih aman untuk komputer laboratorium yang dipakai bergantian.
- Semua teks dari server dimasukkan ke halaman lewat `textContent`, tidak lewat `innerHTML`, untuk mencegah XSS dari data mahasiswa.
- Galat dari API ditampilkan bersama `request_id`. Respons 401 membuat antarmuka keluar otomatis.
- Menu Kelola disembunyikan untuk asisten, tetapi pembatasan sebenarnya tetap ada di API (403).
- Tampilan mengikuti tema terang atau gelap sistem operasi dan bisa dipakai di layar ponsel.

Pengujian antarmuka dilakukan dengan `scripts/evidence/screenshot_webapp.py`. Skrip ini mengisi data contoh lewat API, lalu menjalankan Microsoft Edge tanpa tampilan untuk login, mengisi kehadiran, membuka setiap menu, dan menyimpan tangkapan layar. Skrip yang sama dijalankan terhadap layanan di Cloud Run pada 26 Oktober 2026.

6. Infrastruktur sebagai kode

6.1 Berkas Terraform

Tabel 8. Isi folder infra/terraform

Berkas	Isi
`main.tf`	Provider, API yang diaktifkan, Artifact Registry, Cloud SQL, bucket laporan, Secret Manager, Cloud Run, akun layanan
`cicd.tf`	Workload Identity Pool, provider OIDC GitHub, akun deployer
`monitoring.tf`	Kanal notifikasi surel, uptime check, alert uptime, dasbor
`slo.tf`	Layanan kustom, dua SLO, lima alert yang aktif bila `enable_sla_alerts = true`
`iaas.tf`	VPC, subnet, Cloud Router dan NAT, firewall IAP, VM operasi, disk, jadwal snapshot, bucket cadangan
`variables.tf`, `outputs.tf`	Variabel dengan deskripsi biaya, keluaran URL dan nama sumber daya
`templates/ops-startup.sh.tftpl`	Skrip awal VM: memasang klien PostgreSQL dan Cloud SQL Auth Proxy, menjadwalkan `pg_dump` 02.30 WIB

6.2 Variabel yang mengendalikan biaya

Tabel 9. Variabel boolean untuk komponen berbayar

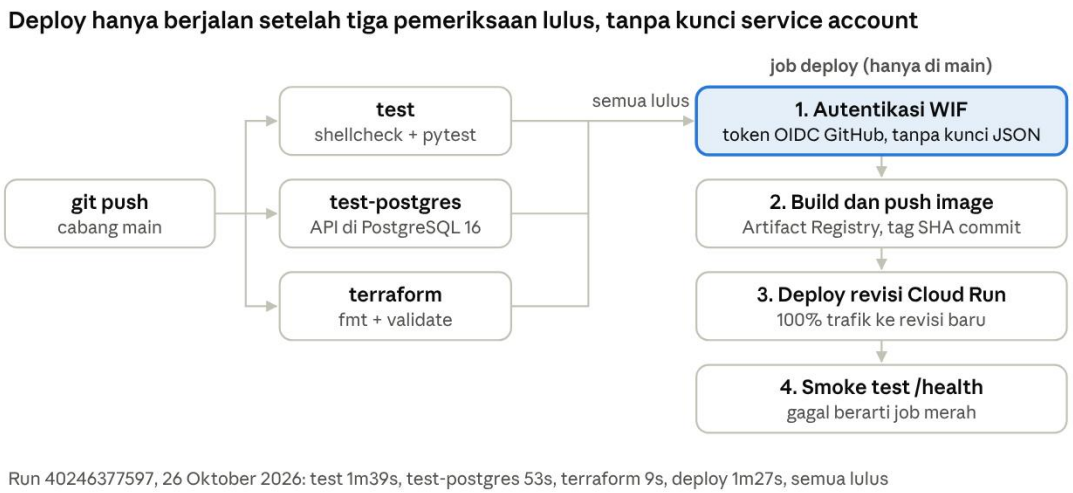
Variabel	Bawaan	Yang dibuat bila `true`	Perkiraan biaya
`enable_sla_alerts`	`false`	SLO fast burn, rasio 5xx, CPU, disk, dan koneksi Cloud SQL	Alert tidak ditagih; hanya notifikasi surel
`enable_ops_vm`	`false`	VM e2-micro, NAT, snapshot, bucket cadangan	US$0 untuk mesin di us-central1 (kuota gratis), sekitar US$1 untuk NAT, beberapa sen untuk snapshot dan cadangan

6.3 Perintah dasar

```
cd infra/terraform
terraform init
terraform fmt -recursive && terraform validate
terraform plan -out tfplan        # tfplan berisi rahasia; jangan di-commit
terraform apply tfplan
terraform output
```

Pada proyek kosong, `apply` membuat 62 sumber daya. Setelah itu, `plan` hanya menunjukkan selisih kecil pada blok `scaling` Cloud Run tingkat layanan, yang tidak berpengaruh.

7. CI/CD



Gambar 3. Alur pipeline GitHub Actions

Tabel 10. Job di .github/workflows/ci-cd.yml

Job	Kapan	Isi
`test`	Setiap push dan pull request	shellcheck untuk `scripts/ops`, pytest dengan SQLite dan laporan cakupan
`test-postgres`	Setiap push dan pull request	pytest terhadap layanan PostgreSQL 16 di runner

Job	Kapan	Isi
`terraform`	Setiap push dan pull request	`terraform fmt -check` dan `validate` tanpa backend
`deploy`	Push ke `main`, setelah tiga job lulus	Autentikasi WIF, build dan push citra bertag SHA, `gcloud run deploy`, smoke test `/health`

Cabang `main` dilindungi: tiga pemeriksaan pertama wajib lulus. Variabel repositori yang dipakai job deploy adalah `GCP_PROJECT_ID`, `GCP_REGION`, `GCP_WIF_PROVIDER`, dan `GCP_DEPLOYER_SA`. Tidak ada rahasia berbentuk kunci yang disimpan di GitHub.

8. Operasi

Semua skrip operasi ada di `scripts/ops/` dan membaca `PROJECT_ID`, `REGION`, serta `ZONE` dari variabel lingkungan. Setiap skrip mencetak rencana tindakan sebelum mengubah apa pun, dan beberapa menulis catatan ke `docs/bukti/013/log/`.

```
export PROJECT_ID=labtrack-1026146 REGION=asia-southeast2 ZONE=us-central1-a
```

Tabel 11. Skrip operasi

Skrip	Pemakaian	Fungsi
`deploy-image.sh`	`deploy-image.sh [--build] <tag>`	Build opsional dengan Cloud Build, deploy revisi, cek `/health`
`rollback.sh`	`rollback.sh [revisi]` atau `--to-latest`	Mengalihkan 100% trafik ke revisi sebelumnya atau yang disebut
`scale.sh`	`scale.sh <min> <max>`	Mengubah batas instans Cloud Run
`rotate-secret.sh`	`rotate-secret.sh`	Versi baru kunci JWT, deploy ulang; semua token lama tidak berlaku
`sql-start-stop.sh`	`start`, `stop`, `status`	Menyalakan atau mematikan Cloud SQL untuk menghemat biaya
`sql-backup-now.sh`	`sql-backup-now.sh ["keterangan"]`	Cadangan Cloud SQL sesuai permintaan
`sql-restore-test.sh`	`sql-restore-test.sh`	`pg_dump` terbaru dipulihkan ke basis data sementara, jumlah baris dibandingkan, basis data sementara dihapus
`vm-start-stop.sh`	`start`, `stop`, `status`	Menyalakan atau mematikan VM operasi
`vm-ssh.sh`	`vm-ssh.sh [perintah]`	SSH lewat IAP
`vm-patch.sh`	`vm-patch.sh [--snapshot] [--reboot]`	Snapshot, pembaruan paket, reboot bila perlu, verifikasi, catat di change log
`vm-snapshot-now.sh`	`vm-snapshot-now.sh [akhiran]`	Snapshot disk sesuai permintaan
`vm-report.sh`	`vm-report.sh`	Inventaris VM: OS, CPU, memori, disk, jaringan, layanan

8.1 Jadwal rutin

Tabel 12. Jadwal otomatis dan manual

Kegiatan	Jadwal	Pelaksana
Cadangan otomatis Cloud SQL	Setiap hari 01.00 WIB	Google
`pg_dump` ke bucket cadangan	Setiap hari 02.30 WIB	Cron di VM operasi
Snapshot disk VM	Setiap hari 03.00 WIB, disimpan 7 hari	Kebijakan snapshot

Kegiatan	Jadwal	Pelaksana
Uji pemulihan	Sebulan sekali	Pengelola, `sql-restore-test.sh`
Patch OS VM	Sebulan sekali atau saat ada pembaruan keamanan	Pengelola, `vm-patch.sh --snapshot`
Rotasi kunci JWT	Setiap semester atau bila diduga bocor	Pengelola, `rotate-secret.sh`
Laporan SLA	Awal bulan	Pengelola, `scripts/sla_report.py`

9. Pemantauan dan SLA

Tabel 13. SLO

SLO	Definisi	Target
Ketersediaan	Permintaan Cloud Run bukan 5xx dibagi seluruh permintaan	99,5% per 30 hari bergulir
Latensi	Permintaan dengan latensi di bawah 800 ms dibagi seluruh permintaan	95% per 30 hari bergulir

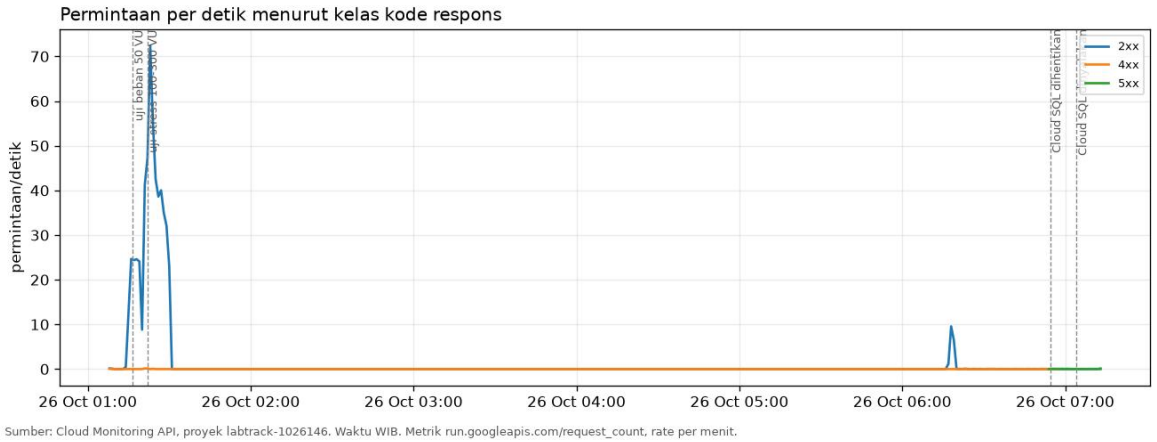
Tabel 14. Kebijakan alert

Nama	Kondisi
LabTrack API is down	Uptime check `/health` gagal dari lebih dari satu region pemeriksa
LabTrack SLO: availability fast burn	Anggaran galat ketersediaan terbakar lebih dari 10 kali laju normal dalam 1 jam
LabTrack API: 5xx ratio	Rasio 5xx di atas 5% selama 5 menit
LabTrack Cloud SQL: CPU	CPU di atas 80% selama 10 menit
LabTrack Cloud SQL: disk	Pemakaian disk di atas 80%
LabTrack Cloud SQL: connections	Jumlah koneksi mendekati batas 25

Laporan SLA dibuat dengan:

```
python scripts/sla_report.py --project labtrack-1026146 --month 2026-10 \
       --uptime-check-id "$(terraform -chdir=infra/terraform output -raw uptime_check_id)"
```

Skrip menghitung ketersediaan berbasis permintaan dan berbasis waktu, anggaran galat terpakai, dan p95, lalu menulis berkas Markdown di `docs/laporan-sla/`.



Gambar 4. Permintaan per kelas respons di Cloud Monitoring, 26 Oktober 2026

10. Pemulihan bencana

Tabel 15. Target pemulihan

Ukuran	Target	Dasar
RPO (data yang boleh hilang)	24 jam	Cadangan otomatis Cloud SQL harian ditambah `pg_dump` harian di bucket terpisah
RTO (lama layanan boleh mati)	2 jam	Waktu menyalakan atau memulihkan Cloud SQL ditambah verifikasi

Tabel 16. Skenario dan langkah pemulihan

Skenario	Langkah
Rilis baru bermasalah	`rollback.sh`; trafik kembali ke revisi sebelumnya dalam hitungan detik
Cloud SQL berhenti	`sql-start-stop.sh start`; tunggu sekitar 11 menit; cek `/health`
Data terhapus tidak sengaja	Pulihkan cadangan otomatis Cloud SQL ke instans baru, atau `pg_dump` terakhir ke basis data sementara, lalu salin baris yang hilang
Kunci JWT bocor	`rotate-secret.sh`; semua pengguna harus login ulang
VM operasi rusak	Buat ulang dari Terraform; disk data dipertahankan, atau pulihkan dari snapshot
Proyek terhapus	Buat proyek baru, `terraform apply`, pulihkan `pg_dump` terakhir dari bucket cadangan



scripts/ops/sql-restore-test.sh | docs/bukti/013/restore-verification-20261026-0625.md

```
# Laporan Verifikasi Restore Cloud SQL

| Butir | Nilai |
|---|---|
| Waktu uji (WIB) | 2026-10-26 06:53 |
| Proyek | labtrack-1026146 |
| Instans sumber | labtrack-db-f6f7 |
| Backup yang dipulihkan | ID 1792970383825, jendela backup 2026-10-25T23:19:43.825Z |
| Instans uji sementara | labtrack-restore-test-202610252325 (dihapus otomatis setelah uji) |
| Durasi total | sekitar 27 menit, memenuhi RTO 2 jam |
| Hasil | **LULUS** |

## Perbandingan jumlah baris

Salinan hasil restore boleh lebih kecil dari produksi karena data yang ditulis setelah jendela backup belum ada di backup. Salinan tidak boleh kosong
bila produksi berisi data, dan tidak boleh lebih besar dari produksi.

| Tabel | Produksi | Hasil restore | Status |
|---|---|---|---|
| users | 2 | 1 | OK |
| students | 12 | 12 | OK |
| courses | 1 | 1 | OK |
| enrollments | 12 | 12 | OK |
| practicum_sessions | 4 | 4 | OK |
| attendances | 48 | 48 | OK |
| lab_reports | 41 | 41 | OK |

Dihasilkan oleh scripts/ops/sql-restore-test.sh.
```

Gambar 5. Hasil uji pemulihan cadangan, 26 Oktober 2026

11. Keamanan

Tabel 17. Kendali keamanan

Ancaman	Kendali
Rahasia bocor dari repositori	Rahasia hanya di Secret Manager; `terraform.tfvars` dan `tfplan` masuk `.gitignore`
Kunci akun layanan dicuri	Tidak ada kunci; GitHub memakai WIF yang dibatasi ke satu repositori

Ancaman	Kendali
Basis data diserang dari internet	Hanya bisa diakses lewat Auth Proxy dengan IAM; tidak ada jaringan yang diotorisasi
Berkas laporan bocor	Bucket privat, tautan 15 menit
Tebak kata sandi dan enumerasi akun	bcrypt; hash tiruan untuk email yang tidak terdaftar
Unggahan berbahaya	Validasi tanda tangan `%PDF-` dan batas 10 MB
SSH terbuka	VM tanpa IP publik; firewall hanya membuka port 22 untuk rentang IAP; jaringan `default` dihapus
XSS di antarmuka web	Semua data ditulis dengan `textContent`

12. Batasan yang diketahui

- Tabel dibuat saat aplikasi mulai dengan `create_all`. Perubahan skema setelah produksi sebaiknya memakai migrasi Alembic.
- Cloud SQL memakai IP publik yang hanya bisa diakses lewat Auth Proxy. IP privat lebih ketat tetapi menambah biaya konektor VPC.
- Menghapus mahasiswa tidak menghapus berkas PDF lamanya di bucket.
- Belum ada endpoint untuk menonaktifkan akun atau mengatur ulang kata sandi; langkahnya ada di runbook dokumen 04 lewat Cloud SQL Studio.
- Probe startup dan liveness memanggil `/health` yang memeriksa basis data. Saat Cloud SQL mati, instans baru gagal naik dan pengguna melihat halaman 503 Cloud Run tanpa `request_id`.
- Anggaran koneksi (3 instans dikali 7, dari 25) ketat. Dua revisi yang berjalan bersamaan saat rilis bisa melewatinya sebentar.